University of Ottawa

CSI 3140 - WWW Structures, Techniques and Standards

Laboratory 4

Client-Server Interaction and Modern Web Services

From SOAP and WSDL to REST, GraphQL, and gRPC

Professor	Mohamed Ali Ibrahim, PhD, PEng
Term	Spring/Summer 2026
Lab Period	Friday, July 3, 2026 to Saturday, July 11, 2026
Submission	Brightspace, by Saturday, July 11, 2026, 11:59 PM
Team Size	2-3 students
Technologies	HTML5, CSS3, JavaScript, Node.js, Express.js, REST API, JSON, OpenAPI/Swagger, Postman or equivalent API testing tool, optional GraphQL or gRPC exploration, browser developer tools

1. Laboratory Overview

The purpose of this laboratory is to introduce students to client-server interaction and modern Web services. Students will build a small Web application where a browser-based client communicates with a server-side API.

This laboratory modernizes the classic Web services topics of JAX-RPC, WSDL, XML Schema, and SOAP by connecting them to today's API technologies: REST, JSON, OpenAPI, GraphQL, and gRPC.

Students will implement a functional REST API and connect it to a front-end Web page using JavaScript. SOAP and WSDL will be covered mainly as historical and enterprise context, while REST and OpenAPI will be the main implementation focus.

The main idea is that older technologies such as SOAP and WSDL are important for understanding legacy enterprise systems, but most modern Web projects use REST + JSON as the default API style. GraphQL and gRPC are introduced as alternatives for more specialized situations.

2. Topics Covered in Lab 4

Topic Area	Required Coverage
Client-server architecture	Browser client, server application, HTTP requests, HTTP responses, endpoints, and data exchange.
REST API design	Resources, routes, HTTP methods, JSON request bodies, JSON responses, and status codes.
Node.js and Express.js	Creating a small server, defining routes, handling requests, and returning JSON responses.
OpenAPI documentation	Documenting API endpoints, request formats, response formats, and status codes.
API testing	Testing endpoints using Postman, Thunder Client, curl, browser tools, or another approved tool.
Front-end API integration	Using JavaScript fetch() to call server endpoints and dynamically update the Web page.

Legacy Web services awareness	Explaining SOAP, WSDL, XML Schema, and JAX-RPC at a high level.
Modern API comparison	Comparing REST, GraphQL, and gRPC using practical examples and trade-offs.
Security and validation	Basic input validation, safe error messages, and awareness of API security concerns.
Deployment thinking	Explaining where the API could be deployed and what would be required for a real deployment.

3. Learning Objectives

- Explain the role of client-server communication in Web applications.
- Build a simple REST API using Node.js and Express.js.
- Design API endpoints using resources and HTTP methods.
- Send and receive JSON data between the browser and the server.
- Use JavaScript fetch() to connect a front-end page to a back-end API.
- Test API endpoints using an API testing tool.
- Document a REST API using an OpenAPI-style description.
- Compare REST with SOAP, GraphQL, and gRPC.
- Explain why SOAP, WSDL, and XML Schema are still found in some enterprise systems.
- Apply basic validation and error handling in an API.
- Prepare a short technical report explaining the API design, testing, and client-server interaction.

4. Problem Description

Each team must create a small client-server Web application for one of the following example topics:

- University event registration system.
- Workshop or mini-course management system.
- Student club activity system.
- Academic service booking system.
- Course resource management system.
- Simple online store or product catalogue.
- Library reservation system.
- Technical conference session manager.
- Any equivalent topic approved by the instructor or TA.

The application must include:

- A front-end Web page built with HTML, CSS, and JavaScript.
- A back-end server built with Node.js and Express.js.
- A REST API that returns and accepts JSON data.
- At least four REST endpoints.
- Client-side JavaScript that calls the API using fetch().
- API testing evidence.
- A short report explaining the implementation and comparing REST with SOAP, GraphQL, and gRPC.

Students may reuse the front-end design from previous laboratories, but the main grading focus of this laboratory is now on API design, client-server communication, REST endpoints, JSON exchange, testing, and documentation.

5. Required Project Structure

Each team must submit a project folder with a clear structure similar to the following:

File or Folder	Required Content
index.html	Main client page that interacts with the API.
styles.css	External stylesheet used to style the client page.
script.js	Front-end JavaScript file that uses fetch() to call the API.
server.js	Node.js/Express server file containing the API endpoints.
package.json	Node.js project configuration and dependencies.
data/	Optional folder containing JSON data files if used.
api-docs/	API documentation, such as an OpenAPI YAML/JSON file or a clearly formatted API table.
screenshots/	Screenshots showing the client, API tests, and results.
report.pdf	Short lab report explaining the project, API, testing, and reflection.
README.md	Instructions explaining how to install, run, and test the project.

A suggested folder structure is:

```
lab4-project/
│
├── public/
│   ├── index.html
│   ├── styles.css
│   └── script.js
│
├── data/
│   └── items.json
│
├── api-docs/
│   └── openapi.yaml
│
├── screenshots/
│
├── server.js
├── package.json
├── README.md
└── report.pdf
```

6. Part A - Server Setup with Node.js and Express.js

Students must create a small server-side application using Node.js and Express.js.

The server must:

- Start correctly using npm start or node server.js.
- Listen on a local port such as 3000.
- Serve JSON responses from API endpoints.
- Serve the front-end files or clearly explain how the front-end is opened.
- Use meaningful route names.
- Include comments explaining the main server sections.
- Avoid unnecessary complexity.

Example:

```
const express = require("express");
const app = express();
const PORT = 3000;

app.use(express.json());
app.use(express.static("public"));

app.get("/api/status", function (req, res) {
  res.json({ message: "Lab 4 API is running" });
});

app.listen(PORT, function () {
  console.log(`Server running on http://localhost:${PORT}`);
});
```

7. Part B - REST API Design

The project must include at least four REST API endpoints. The endpoints must demonstrate meaningful use of HTTP methods.

Minimum required endpoints:

Method	Example Endpoint	Purpose
GET	/api/items	Return a list of resources.
GET	/api/items/:id	Return one specific resource.
POST	/api/items	Create or simulate the creation of a new resource.
PUT, PATCH, or DELETE	/api/items/:id	Update, modify, or delete a resource.

The API must:

- Use nouns for resources instead of verbs.
- Use JSON for request and response bodies.
- Return appropriate HTTP status codes.
- Return clear error messages when a resource is not found.
- Validate required input fields for POST, PUT, or PATCH.
- Avoid returning raw server errors to the client.

Example endpoint:

```
app.get("/api/events", function (req, res) {
  res.json(events);
});
```

Example JSON response:

```
[
  {
    "id": 1,
    "title": "REST API Workshop",
    "category": "Workshop",
    "capacity": 30
  },
  {
```

```
    "id": 2,
    "title": "GraphQL Introduction",
    "category": "Seminar",
    "capacity": 25
  }
]
```

8. Part C - Front-End Client Interaction

The client page must communicate with the server API using JavaScript. Students must use fetch() to:

- Request data from the server.
- Display API data dynamically on the Web page.
- Submit form data to the server.
- Display success or error messages based on the server response.
- Handle at least one failed request or invalid input case.

Example:

```
async function loadEvents() {
  const response = await fetch("/api/events");
  const events = await response.json();

  const list = document.querySelector("#event-list");
  list.innerHTML = "";

  events.forEach(function (event) {
    const item = document.createElement("li");
    item.textContent = `${event.title} - ${event.category}`;
    list.appendChild(item);
  });
}

loadEvents();
```

The front-end page must include at least:

- One section that displays data loaded from the API.
- One form that sends data to the API.
- One dynamic update based on the API response.
- One visible error or validation message.

9. Part D - JSON Data and Server-Side Validation

The server must store or simulate data using one of the following:

- A JavaScript array of objects.
- A local JSON file.
- Another simple data source approved by the TA.

A full database is not required for this laboratory. The data must include at least four objects. Each object must include at least four fields.

Example:

```
const events = [
  {
    id: 1,
    title: "REST API Workshop",
    category: "Workshop",
```

```
    date: "2026-07-02",
    capacity: 30
  },
  {
    id: 2,
    title: "SOAP and WSDL Legacy Systems",
    category: "Lecture",
    date: "2026-07-04",
    capacity: 40
  }
];
```

Server-side validation must check at least:

- Required fields are present.
- At least one field follows a specific rule.
- Invalid requests return a 400 Bad Request response.
- Requests for missing resources return a 404 Not Found response.

Example:

```
app.post("/api/events", function (req, res) {
  const { title, category, date } = req.body;

  if (!title || !category || !date) {
    return res.status(400).json({
      error: "Title, category, and date are required."
    });
  }

  const newEvent = {
    id: events.length + 1,
    title,
    category,
    date
  };

  events.push(newEvent);
  res.status(201).json(newEvent);
});
```

10. Part E - OpenAPI or API Documentation

Each team must document the API. Students may use one of the following formats:

- An OpenAPI YAML or JSON file.
- Swagger UI if configured.
- A clearly formatted API documentation table in the report.

The documentation must include:

- API title and short description.
- Base URL.
- List of endpoints.
- HTTP method for each endpoint.
- Request body format where applicable.
- Response format.
- Possible status codes.
- Example request and response.

Example documentation table:

Method	Endpoint	Description	Success Response	Error Response
GET	/api/events	Returns all events.	200 OK	None
GET	/api/events/:id	Returns one event.	200 OK	404 Not Found
POST	/api/events	Creates an event.	201 Created	400 Bad Request
DELETE	/api/events/:id	Deletes an event.	200 OK	404 Not Found

Students should understand that OpenAPI plays a role similar to a modern contract for REST APIs, while WSDL played that role for SOAP-based services.

11. Part F - Legacy Web Services Awareness: SOAP, WSDL, XML Schema, and JAX-RPC

Students are not required to implement SOAP in this laboratory. However, the report must include a short explanation of the legacy Web service stack.

The report must explain:

- What SOAP is.
- What WSDL is.
- What XML Schema is.
- What JAX-RPC was used for.
- Why these technologies are considered legacy for most new Web applications.
- One example of where SOAP and WSDL may still be used today.

Students must also include a short comparison between SOAP/WSDL and REST/OpenAPI.

Aspect	SOAP/WSDL	REST/OpenAPI
Message format	XML	Usually JSON
Contract	WSDL	OpenAPI
Style	Operation-based	Resource-based
Complexity	Higher	Lower
Common use today	Enterprise and government systems	Public APIs and modern Web applications

12. Part G - Modern API Comparison: REST, GraphQL, and gRPC

The report must include a short comparison of REST, GraphQL, and gRPC. Students must explain:

- Why REST is a good default for many Web APIs.
- When GraphQL can be useful.
- When gRPC can be useful.
- Which API style would be most appropriate for their Lab 4 project and why.

API Style	Main Format	Best Use Case	Main Strength
REST + OpenAPI	JSON over HTTP	Public and CRUD-style APIs	Simple and widely supported
GraphQL	JSON over HTTP	Client-driven data retrieval	Flexible queries
gRPC	Protocol Buffers over HTTP/2	Internal microservices	Fast and strongly typed

For implementation, students are required to build a REST API. GraphQL and gRPC are comparison topics only, unless the team receives approval for an optional extension.

13. Part H - API Testing Requirements

Each team must test the API before submission. Testing must include:

- Testing each endpoint with valid requests.
- Testing at least two invalid requests.
- Testing a request for a missing resource.
- Checking returned status codes.
- Checking returned JSON data.
- Testing the front-end page with the running server.
- Checking the browser console for errors.
- Checking the server terminal for errors.

Students may use Postman, Thunder Client, curl, browser developer tools, or any equivalent API testing tool approved by the TA.

The report must include screenshots showing:

- A successful GET request.
- A successful POST request.
- An invalid request returning an error.
- A front-end page displaying data received from the API.

14. Part I - Basic Security and Error Handling

Students must show basic awareness of API security and error handling.

The project must:

- Validate input before accepting it.
- Avoid exposing raw internal errors.
- Return clear but safe error messages.
- Use appropriate status codes.
- Avoid trusting client-side validation only.
- Explain in the report what additional security would be needed in a real production API.

The report should briefly mention at least three real-world API security concerns, such as authentication, authorization, HTTPS, input validation, rate limiting, CORS configuration, and sensitive data protection. Full authentication is not required for this laboratory.

15. Client-Server and Modern Web Services Checklist

Each team must complete the following checklist and include it in the report.

Item	Completed?
The project includes a clear client-server structure.	
The server runs using Node.js and Express.js.	
The server provides at least four REST API endpoints.	

The API uses JSON request and response bodies.	
The API uses appropriate HTTP methods.	
The API uses appropriate HTTP status codes.	
The API includes server-side validation.	
The API returns clear error messages for invalid requests.	
The client uses fetch() to call the API.	
The client dynamically displays data received from the API.	
The client includes a form that sends data to the API.	
API testing screenshots are included.	
The report includes API documentation or an OpenAPI-style description.	
The report explains SOAP, WSDL, XML Schema, and JAX-RPC.	
The report compares SOAP/WSDL with REST/OpenAPI.	
The report compares REST, GraphQL, and gRPC.	
The project was tested in a browser.	
The browser console and server terminal were checked for errors.	
A README file explains how to run the project.	

16. Laboratory Demonstration Requirement

Each team must demonstrate the laboratory work to the TA. The report will be considered for grading only if the demonstration has been completed. All team members must be present during the demonstration.

During the demonstration, the TA may ask any team member to:

- Start the server.
- Explain the project structure.
- Explain each API endpoint.
- Send a test request to the API.
- Explain the JSON request and response format.
- Show how the front-end calls the API.
- Explain server-side validation.
- Explain how errors are handled.
- Compare REST with SOAP, GraphQL, and gRPC.
- Explain the role of OpenAPI documentation.

Each student must be able to explain the team submission clearly.

17. Testing Requirements

Each team must test the application before submission. Testing must include:

- Running the server locally.
- Opening the front-end page in a modern browser.

- Checking that CSS and JavaScript files load correctly.
- Testing all REST endpoints.
- Testing valid and invalid API requests.
- Testing front-end interaction with the API.
- Checking that JSON data is displayed correctly.
- Checking that invalid input produces a clear error message.
- Checking the browser console for errors.
- Checking the server terminal for errors.
- Testing the application at desktop, tablet, and mobile widths if the front end is responsive.
- Reviewing the API documentation for consistency with the implemented endpoints.

18. Deliverables

Each team must submit one ZIP file on Brightspace containing:

- All HTML files used for the client.
- All CSS files used for styling.
- All front-end JavaScript files.
- The Node.js server file.
- The package.json file.
- Any JSON data files used by the project.
- API documentation or OpenAPI file.
- A short lab report in PDF format.
- Screenshots showing the client page in a browser.
- Screenshots showing successful API requests.
- Screenshots showing invalid API requests and error responses.
- Screenshots showing the front end displaying data from the API.
- Screenshots or notes showing testing evidence.
- A README file explaining how to install, run, and test the project.

The ZIP file must not include the node_modules folder.

19. Report Requirements

The lab report must include:

- Course code, lab number, team members, and student numbers.
- A short description of the project topic.
- A file structure diagram or list.
- Instructions explaining how to run the server and open the client.
- A description of each REST endpoint.
- A table documenting the API.
- Examples of JSON requests and responses.
- A description of the front-end and back-end interaction.
- A description of the validation and error handling rules.
- API testing screenshots.
- A short explanation of SOAP, WSDL, XML Schema, and JAX-RPC.
- A comparison between SOAP/WSDL and REST/OpenAPI.
- A comparison between REST, GraphQL, and gRPC.
- The completed checklist.

- A short reflection on problems encountered and how they were solved.

20. Grading Rubric

Component	Points
Correct project structure, README, and ability to run the application	10
Node.js/Express server setup and organization	10
REST API design: endpoints, resources, HTTP methods, and JSON responses	15
Front-end integration using fetch() and dynamic display of API data	15
Server-side validation, error handling, and appropriate status codes	10
API documentation or OpenAPI-style description	10
API testing evidence, screenshots, and debugging	10
Legacy Web services awareness: SOAP, WSDL, XML Schema, and JAX-RPC	5
Modern API comparison: REST, GraphQL, and gRPC	5
Laboratory demonstration to the TA with all team members present	3
Report quality, clarity, screenshots, checklist, and reflection	7
Total	100

21. Important Notes

- Late submissions will not be accepted unless an official accommodation or approved exceptional circumstance applies.
- All team members must understand the submitted client code, server code, API endpoints, and report.
- External code, templates, images, fonts, icons, or resources must be cited if used and must be permitted for academic use.
- Students must not submit a project generated entirely by an external tool without understanding, adapting, and documenting the work.
- Academic integrity rules apply to all submitted files and reports.
- A full database is not required for this laboratory.
- Authentication is not required, but students must discuss what authentication or authorization would be needed in a real API.
- The node_modules folder must not be submitted.
- The REST API must be implemented manually using Node.js and Express.js. Large frameworks or API generators are not permitted unless explicitly approved.
- GraphQL and gRPC are not required for implementation. They are required for comparison in the report.

22. Timeline

Date	Suggested Work
Monday, June 29	Read the lab statement, choose the project topic, and plan the client-server structure.
June 30 - July 1	Create the Node.js project, install Express, and build the initial server.
July 2 - July 3	Implement the main REST API endpoints and JSON data structure.
July 4 - July 5	Connect the front-end page to the API using fetch().
July 6	Add server-side validation and error handling.
July 7	Create API documentation or an OpenAPI-style endpoint table.
July 8	Test the API using Postman, Thunder Client, curl, or another tool.
July 9	Complete the report sections on SOAP/WSDL, REST/OpenAPI, GraphQL, and gRPC.
July 10	Final testing, debugging, screenshots, and README preparation.
Saturday, July 11	Submit the ZIP file on Brightspace by 11:59 PM.